How Much Junk Do Rules Catch? Evaluating and Tuning pygarble, a Dependency-Free Gibberish Screener, on Survey Responses and Social-Media Text

Ujjwal Singh Rao
ujjwalsrao@gmail.com

Ricky Singh
rickragv@gmail.com

October 2026
https://github.com/brightertiger/pygarble

Abstract

Text that reaches a language model, a survey database or a training corpus is not always text. Keyboard mashing, random strings and placeholder answers are common, and the usual detectors for them are either trained classifiers with model downloads or hand-written rules that are rarely measured. pygarble is a Python library with no runtime dependencies that screens text with 32 rule-based and statistical strategies combined into named profiles. We evaluate it on two public English short-text datasets, a Kaggle collection of 3,513 free-text survey responses labelled as junk and a Hugging Face collection of 9,971 English rows that pairs 4,000 random strings with 5,971 tweets, against a character Markov-chain gibberish detector and a language identifier, every method at its own default decision. Out of the box, the `english_fusion` profile captures 89.5 % of the survey junk and recalls 99.1 % of the random strings at a precision of 0.906, from a 1.05 MB install that processes a thousand short texts in 62.0 ms; no baseline captures more survey junk while also flagging fewer tweets. A small labelled sample shifts the trade-off: a weight for every strategy and a threshold learned with Optuna from 20 % of the rows, in about 17 seconds of search, cuts the false-positive rate on tweets from 6.9 % to 4.9 % and raises precision from 0.906 to 0.930 on the held-out 80 %, with capture about level (89.7 % against 89.6 %) and recall lower (97.6 % against 99.1 %; means over five draws). The learned weights are interpretable at the extremes: word lookup gets full weight in every draw, and the two encoding checks, which fire on tweets this dataset labels clean, get almost none. A dependency-free rule ensemble is therefore competitive on real junk as shipped and adapts to a domain in seconds, with every decision explainable strategy by strategy. Code, per-row predictions and a verification script are released so that every number can be recomputed.

1 Introduction

Free-text fields collect junk. Respondents mash the keyboard to reach the next page of a survey, bots submit random strings to sign-up forms, and scraped text carries encoding damage. Each of these costs something downstream: a coder reads it, a model is trained on it, or an analysis counts it as an answer. The problem has become more visible with large language models (LLMs), where unreadable input wastes tokens, degrades retrieval, and in guardrail settings is itself a signal that something upstream has gone wrong. The authors' own motivation for pygarble is exactly this guardrail role: a cheap, local check that runs on every prompt and every model output before anything more expensive does.

Two families of tools exist for the task. Trained classifiers, such as the DistilBERT gibberish model used in open-source LLM guardrail frameworks [6], are accurate on the distribution they were trained on but need a model download and a deep-learning runtime, and their decisions are hard to explain row by row. Rule-based detectors, from character Markov models [7] to the nonsense evaluator Nostril [5] and the quality heuristics used to filter web corpora [8, 9], are cheap, local and explainable, and are often the first filter a team writes by hand. What is rarely reported is how much junk such rules actually catch on real data, and how much good text they wrongly flag.

This paper measures one such library. pygarble [10] packages 32 strategies, each a function from a string to a score in $[0, 1]$, together with voting rules that combine them into profiles. It has no runtime dependencies and ships its English reference tables inside the wheel. We ask two plain questions. First, at default settings, on data that played no part in setting its defaults, how much junk does it capture, how much legitimate text does it flag, and what does it cost compared with two common alternatives? Second, when a user has a small labelled sample of their own data, how far can the same strategies be tuned to it, and what do the learned weights say about which signals matter?

The contributions are:

- A benchmark on two public datasets of short text, one of real human junk from survey research and one of random strings against tweets, with a frozen protocol, saved per-row predictions and a script that recomputes every reported number.
- A comparison of pygarble's default and `english_fusion` profiles with a character Markov-chain detector and a language identifier, all at their default decisions, on capture rate, precision, recall, false-positive rate, speed and install size.
- A tuning study showing that a weight per strategy and a threshold, learned with Optuna from 20 % of the rows in seconds, lower the false-positive rate and raise precision on the held-out 80 % in every draw, at the cost of some recall and with capture about level on average; the learned weights are reported in full, and the study is evaluated against the two profiles on identical rows and reported separately from the main results.
- A description of pygarble's strategy families and profiles, with the full strategy list in the appendix.

2 pygarble

pygarble targets Python 3.8 and later and has no runtime dependencies. The installed package, including its embedded English tables, is 1.05 MB. Besides gibberish detection it scans text for secrets, personal data and profanity and can redact them; those features are outside this paper.

2.1 Strategies

A *strategy* is a function from a string to a score in $[0, 1]$ and a binary decision, with a flag that says whether the strategy applies to the input at all. A strategy that needs letters or a minimum amount of text, such as the Markov chain or cross parsing, reports that it does not apply when the input has too little of either, rather than returning a low score, so that absence of evidence is not counted as evidence that the text is clean. The 32 strategies fall into six families (Table 1; the full list with one-line descriptions is in Appendix B).

Character statistics measure properties of the character stream that English rarely violates: the vowel ratio, Shannon entropy, the longest consonant run, the letter-frequency profile, the symbol share, and whether tokens are pronounceable.

English character models compare the text with tables learned from English: a character Markov chain over novel tokens, bigram and trigram probabilities, a rare-trigram count, and a log-likelihood ratio of English against a uniform reference.

Table 1: Strategy families in pygarble. Counts sum to 32.

Family	Count	What it detects	Examples
Character statistics	8	Letter mixes English rarely produces	`vowel_ratio`, `symbol_ratio`
English character models	5	Unlikely letter sequences	`markov_chain`, `rare_trigram`
Word-level checks	6	Unknown words, missing English word patterns	`word_lookup`, `word_anomaly`
Structure and pattern checks	6	Repeats, keyboard mash, hex, local corruption	`repetition`, `hex_string`
Encoding and script checks	3	Mojibake, control characters, spoofing	`mojibake`, `unicode_script`
English-reference window statistics	4	Distance from synthetic English	`cross_parsing`, `ngram_rank`

Word-level checks look words up in an embedded dictionary and measure the unknown fraction, the fraction of anomalous tokens, the density of English function words, the presence of English affixes, the plausibility of adjacent word pairs, and whether the word-frequency profile conforms to Zipf's law.

Structure and pattern checks target forms of junk that are not statistical: repeated substrings, keyboard-row walks and adjacent-key sequences, hexadecimal strings, built-in patterns such as a letter repeated four or more times or a five-key keyboard-row walk (with optional user-supplied patterns), and local anomalies inside otherwise ordinary text.

Encoding and script checks fire on damage rather than on meaning: mojibake byte signatures left by a wrong decoding, control characters, and mixed or confusable Unicode scripts.

English-reference window statistics compare a window of the text with frequent English rather than applying a hand-written rule: cross parsing [14], primed compression, n-gram rank [3] and a permutation test. Each statistic is measured against synthetic English of similar length, and a score of 0.5 sits beyond the 99th percentile of that reference.

2.2 Ensembles, voting and profiles

`EnsembleDetector` runs a list of strategies and combines them under a voting rule. With `any` voting, one applicable member firing flags the text; `all`, `majority`, `average` and `weighted` voting behave as their names say. `fisher` voting treats each member's score as a test statistic: a table shipped with the package records what that member scores on synthetic English word salads of 8 to 480 characters, the member's p-value is the smallest tail probability whose threshold the score exceeds, and the p-values are combined with Fisher's method [4]. The combined p-value is mapped to a score so that the default threshold of 0.5 flags a combined p-value of 0.001 or less. Named member lists are called *profiles*; Appendix A lists all seven. Two are measured here:

- `english`, the default: Markov chain, log-likelihood ratio, word anomaly, mojibake, keyboard adjacency and control characters, under `any` voting. One member firing is enough.
- `english_fusion`, opt-in: word lookup, log-likelihood ratio and cross parsing under `fisher` voting. Each member's p-value is floored at 0.001, so when two or more members apply, at least two must score beyond their null thresholds before a text is flagged; one member decides alone only when it is the only one that applies.

A user can also pass any list of strategies, with a weight for each under `weighted` voting, which is how the tuned configuration in Section 5.3 is built. The English profiles score English plausibility; they are not language identifiers, and meaningful text in other Latin-script languages can be flagged. Scores are heuristic, not calibrated probabilities.

3 Datasets

3.1 Survey Junk

The Kaggle dataset *Gibberish Text Classification* [13] (hereafter Survey Junk) was compiled by a market-research practitioner from free-text responses of poor-quality survey respondents. Its junk file holds 3,767 rows, every one labelled as junk; despite its `.xls` extension it is a Windows-1252 CSV. We strip each response, drop empty rows and drop exact duplicates, leaving 3,513 rows. Typical rows are keyboard mashes such as runs of adjacent keys, repeated letters, and letters mixed with digits. The median response is 11 characters long and 69.5 % are a single token.

Because the file has no clean rows, the only metric on this dataset is the *capture rate*: the share of rows a method flags. The licence is listed as unknown on Kaggle, so the repository stores row indices and the SHA-256 hash of each file, never the text.

3.2 Tweets vs. Random Strings

The Hugging Face dataset *gibberish_multilingual* [11] (hereafter Tweets vs. Random Strings; Apache-2.0) pairs random character strings with social-media posts in seven languages. We pool its train, validation and test files and keep the 9,971 non-empty rows whose language field is English: 4,000 random strings, labelled junk, and 5,971 tweets, labelled clean. Labels are used exactly as given and duplicates are kept (215 rows repeat an earlier text). The random strings are lower-case letter sequences such as `edkylxhboa`; the tweets carry hashtags, handles and, in 26.4 % of cases, mojibake byte signatures from UTF-8 text, mostly emoji and punctuation, decoded as Latin-1. The dataset labels those tweets clean because it was built to separate random strings from posts.

On this dataset we report precision, recall and the false-positive rate (FPR), the share of tweets flagged.

4 Experimental setup

Methods. pygarble 0.11.0 is used with its defaults frozen at commit `e3135ed` (results were produced at `72b5b2e`, whose package code is identical); no default or threshold was changed after any result was seen. Two baselines run on CPU with pinned versions:

- **gibberish-detector** 0.1.1 [7] (the Markov baseline below), a widely used character Markov model on PyPI. The package ships no model, so one is trained with its own trainer on the corpus its README uses, and a text is flagged when its score exceeds the command-line default of 4.0.
- **lingua** 2.2.0 [12], a language identifier, under the policy "not detected as English is junk".

There are no neural baselines; the DistilBERT gibberish model was excluded to keep the comparison to tools that run with no model download.

Decisions. Every method is scored at its own default decision, the binary output it gives with default settings. The question is what each tool does as installed. The baselines expose no tuning step short of retraining the Markov model, and the tuning study in Section 5.3 measures how far pygarble can be adapted with a small sample, not whether it beats a re-tuned alternative. In the main comparison no threshold is fitted on either dataset.

Intervals. Uncertainty is given as 95 % percentile intervals from 1,000 bootstrap resamples of rows, drawn with a fixed seed and shared by all methods.

Pilot and protocol changes. Before the protocol was written, both datasets were scored once with pygarble's defaults to judge feasibility. The `english_fusion`

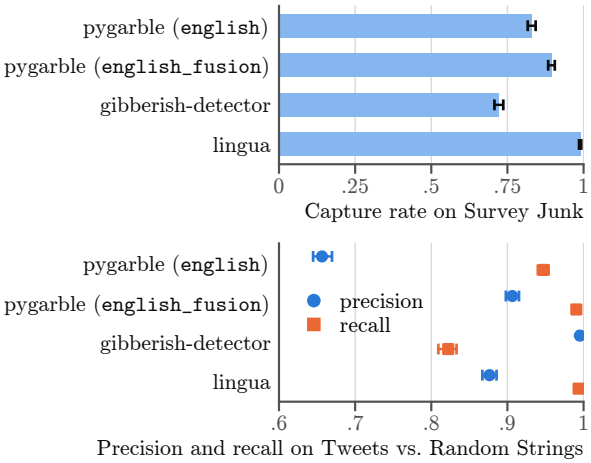


Figure 1: Top: capture rate on Survey Junk (3,513 responses labelled as junk). Bottom (axis starts at 0.6): precision and recall on Tweets vs. Random Strings (4,000 random strings, 5,971 tweets). Every method at its default decision, with 95 % bootstrap intervals.

numbers from that pilot equal the ones reported here, because nothing in the package changed afterwards. The first version of the protocol also fitted thresholds at fixed false-positive rates on a development split of the tweets dataset and compared more baselines and pygarble configurations; both were computed and then removed from the paper's scope (Section 7). The tuning study in Section 5.3 was added after the main results were seen. It learns from labelled rows of both datasets and is evaluated only on rows it did not see. Rows with identical text are kept on the same side of the split, so no tweet in the held-out set also appears in the tuning sample. Its threshold range was widened once, from a lower bound of 0.2 to 0.05, and its budget raised from 400 to 1,000 trials, because every draw of a first run chose the bound and found its best trial late. That decision rested on the tuning samples; the first run's held-out results had also been computed, and they are kept in the repository history. The tuning sample was at first drawn row by row, which let copies of a repeated tweet fall on both sides of the split; the study was re-run with the grouped split, and the earlier results, which were slightly more favourable to the tuned configuration, are recorded in the deviations log.

Environment. All runs are single-threaded on an Apple M2 with Python 3.12.2. Hardware, package versions and the pygarble commit are recorded with the results.

5 Results

5.1 Survey Junk

Table 2 and Figure 1 give the capture rates. `english_fusion` flags 89.5 % of the junk responses

Table 2: Capture rate on Survey Junk: the share of 3,513 junk responses each method flags at its default decision, with 95 % bootstrap intervals. The two pygarble rows are the same library run with different profiles, whose names are set in typewriter. Lingua flags a text when the detected language is not English. * best, ** second best per column.

Method	Capture rate	95% CI
pygarble (`english`)	0.830	[0.817, 0.843]
pygarble (`english_fusion`)	0.895**	[0.884, 0.905]
gibberish-detector	0.722	[0.707, 0.736]
lingua	0.989*	[0.985, 0.992]

Table 3: Precision, recall and false-positive rate on Tweets vs. Random Strings (4,000 random strings as junk, 5,971 tweets as clean), every method at its default decision; each cell shows the value with its 95 % bootstrap interval beneath. FPR is the share of tweets flagged. The two pygarble rows are the same library run with different profiles, whose names are set in typewriter. Lingua flags a text when the detected language is not English. * best, ** second best per column.

Method	Precision	Recall	FPR
pygarble	0.656	0.947	0.332
(`english`)	[0.645, 0.670]	[0.940, 0.954]	[0.320, 0.344]
pygarble	0.906**	0.991**	0.068**
(`english_fusion`)	[0.898, 0.915]	[0.987, 0.993]	[0.062, 0.075]
gibberish-detector	0.995*	0.822	0.003*
	[0.993, 0.997]	[0.809, 0.833]	[0.002, 0.004]
lingua	0.876	0.994*	0.094
	[0.867, 0.886]	[0.991, 0.996]	[0.087, 0.101]

(0.884–0.905), and the default `english` profile 83.0 %. The Markov baseline captures 72.2 %, and lingua, which flags anything it does not identify as English, captures 98.9 %.

The ordering follows what each method can see. Keyboard mashes and random letters have improbable character transitions and no dictionary words, so every method catches most of them. Lingua's near-complete capture comes from the policy rather than from junk detection: a short mash is not confidently English, so it is flagged, and Section 5.2 shows what that policy costs on real text. The Markov baseline misses the rows whose transitions happen to be plausible, such as repeated syllables. The 369 rows that `english_fusion` misses are mostly one to three characters long, letters mixed with digits, strings with no letters at all, or a short syllable or punctuation mark repeated; some are real words or short phrases, which read as English to character and word statistics.

The dataset is used as labelled, and it has some label noise: 15 rows are flagged by none of the four methods, and on inspection many are meaningful low-effort answers such as a thanks or a short positive phrase. The capture rates therefore have a ceiling below one.

5.2 Tweets vs. Random Strings

Table 3 gives precision, recall and false-positive rate. The random strings are easy for every method: `english_fusion` recalls 99.1 % of them and lingua 99.4 %. What separates the methods is how many tweets they flag. `english_fusion` flags 6.8 % of tweets, for a precision of 0.906. The Markov baseline flags only 0.3 %, the lowest of any method, and reaches a precision of 0.995, but at the cost of recalling only 82.2 % of the random strings. Lingua flags 9.4 % of tweets, the posts it does not identify as English. The default `english` profile flags 33.2 % of tweets, the highest rate in the table. Two causes overlap. Its two encoding-damage checks, mojibake and control characters, flag 28.5 % of tweets between them, almost all through the control-character check on the C1 control codes of mis-decoded text; 26.4 % of tweets carry mojibake, which the labels treat as clean. Its Markov-chain member alone flags 17.8 % of tweets, and its four non-encoding members together, Markov included, flag 18.2 %; removing the two encoding checks would still leave that many tweets flagged.

This dataset is a measure of false positives on informal English more than of recall on realistic junk: its junk is random strings, which share almost nothing with English, so any character model separates them.

5.3 Tuning to a use case from a small labelled sample

The profiles above are fixed member lists. A user with a labelled sample of their own data can instead learn how much each strategy should count. We learned a weight in $[0, 1]$ for each of the 32 strategies, where 0 switches a strategy off, and a threshold in $[0.05, 0.80]$, under pygarble's `weighted` voting: a text's score is the weighted mean of the scores of the members that apply to it, and the text is flagged when the score reaches the threshold. Weights move in steps of 0.05 and the threshold in steps of 0.01.

The tuning sample is a stratified 20 % of each dataset: 703 Survey Junk rows and 1,994 Tweets vs. Random Strings rows in proportion to their labels. Rows with identical text are kept on the same side of the split, so no tweet in the held-out set also appears in the tuning sample. Optuna [1] with its tree-structured Parzen estimator sampler [2], with a fixed seed, searched 1,000 configurations for the best F1 on that sample, with junk as the positive class. The search was still improving when it stopped: in every draw the best configuration came after trial 832 of 1,000, so a larger budget may fit the tuning sample better. Every strategy is scored once per row and cached (13 seconds for both datasets), so the search takes about 17 seconds per sample on one core. The draw is repeated with five seeds, 20261011 to 20261015, the first being the primary draw. Each learned configuration was rebuilt as an `EnsembleDetector` and run on the held-out 80 %, where it reproduced the cached decisions on every row, and the two profiles were scored on the same rows.

Table 4 gives the results. On the primary draw the tuned configuration flags 4.0 % of held-out tweets (0.035–0.046) against 7.1 % for `english_fusion` and 33.8 % for `english`. It captures 86.8 % of held-out survey junk against 89.1 %, and recalls 96.7 % of the random strings against 99.0 %. Over the five draws its false-positive rate is 4.9 % (sample standard deviation 1.2 points) against 6.9 %, its capture 89.7 % against 89.6 %, and its recall 97.6 % against 99.1 %. The gain over `english_fusion` is in false positives and precision, which are better in every draw; capture is about equal on average and recall is lower. The tuned capture is below the profile's in two of the five draws and the tuned recall in four, and the primary draw is one where both are lower: its recall interval lies wholly below the profile's. Across the draws, recall was as low as 94.0 % and capture as low as 86.8 %. The tuned rows vary more between draws than the profiles, whose only variation is which rows are held out, because a 20 % sample is small enough for the learned weights to differ from draw to draw.

Between 24 and 31 of the 32 strategies kept a non-zero weight, and the thresholds lie between 0.13 and 0.16. The largest mean weights went to `word_lookup` (1.00), `zipf_conformity` (0.81), `entropy_based` (0.78) and `hex_string` (0.76); the first had full weight in every draw, while the others moved between draws. The smallest went to `ngram_rank` (0.10), `control_characters` (0.04) and `mojibake` (0.03); no strategy was switched off in every draw, and in the primary draw `control_characters`, `pattern_matching` and `mojibake` have weight zero. Learning from the labels alone, the search thus gave almost no weight to the two encoding-damage checks, which fire on tweets that Tweets vs. Random Strings labels clean. Appendix C lists every weight.

A user reproduces this on their own data by scoring a labelled sample with each strategy, searching weights and a threshold (the benchmark's `tune.py` does this with Optuna), and building the detector from the strategies with non-zero weight:

```
EnsembleDetector(strategies=names,
                 weights=weights,
                 voting="weighted",
                 threshold=threshold)
```

The package itself is unchanged; the weights describe these two datasets, not junk in general.

5.4 Cost

Table 5 lists time per 1,000 texts, import time and install size. pygarble scores a thousand short texts in 62.0 ms with `english_fusion` and installs in 1.05 MB with nothing else. The Markov baseline is the fastest, at 6.8 ms, with a 6.51 MB install that includes its training corpus.

Table 4: Tuning from a small labelled sample, evaluated on the held-out 80 % of each dataset. The tuned configuration has a weight per strategy and a threshold learned with Optuna on a stratified 20 % sample; the two profiles run at their defaults on the same held-out rows. Top: the primary draw (2,810 Survey Junk rows; 3,200 random strings and 4,777 tweets), with 95 % bootstrap intervals beneath. Bottom: mean and sample standard deviation over five draws. Intervals for the tuned row hold the learned weights fixed; variation between draws is in the bottom block. In the mean block the capture ranking is within one sample standard deviation. FPR is the share of tweets flagged. The tuned rows use labelled data from both datasets and are post hoc in that sense. * best, ** second best per column within each block.

	Survey Junk	Tweets vs. Random Strings		
Method	Capture	Precision	Recall	FPR
Primary draw (seed 20261011), with 95 % bootstrap interval				
pygarble (`english`)	0.826	0.653	0.949	0.338
	[0.811, 0.840]	[0.639, 0.666]	[0.941, 0.956]	[0.324, 0.351]
pygarble (`english_fusion`)	0.891*	0.904**	0.990*	0.071**
	[0.879, 0.903]	[0.894, 0.913]	[0.987, 0.994]	[0.064, 0.078]
pygarble (tuned weights)	0.868**	0.941*	0.967**	0.040*
	[0.856, 0.882]	[0.934, 0.949]	[0.960, 0.973]	[0.035, 0.046]
Mean ± sample standard deviation over five draws				
pygarble (`english`)	0.830 ± 0.005	0.653 ± 0.002	0.946 ± 0.002	0.337 ± 0.003
pygarble (`english_fusion`)	0.896** ± 0.004	0.906** ± 0.002	0.991* ± 0.001	0.069** ± 0.002
pygarble (tuned weights)	0.897* ± 0.027	0.930* ± 0.016	0.976** ± 0.023	0.049* ± 0.012

Table 5: Footprint. Time is the default decision over all 9,971 texts of Tweets vs. Random Strings on one thread of an Apple M2, median of five runs after a warm-up. Install size counts the installed files of the distribution and its runtime dependencies, plus the training corpus gibberish-detector's model is built from. Import time covers the import statement only. Building each detector and the warm-up run, in which lingua loads its language models, are not timed.

Method	ms/1k	Import (s)	Install (MB)	Deps
pygarble (`english`)	66.4	0.021	1.05	none
pygarble (`english_fusion`)	62.0	0.021	1.05	none
gibberish-detector	6.8	0.039	6.51	none[a]
lingua	1039.0	0.001	307.47	none[b]

[a] Ships no model; trains one from a text corpus at start-up (the benchmark uses the project's `big.txt`, counted in its install size).
[b] Bundles its language models in the wheel.

Lingua needs 1039.0 ms and a 307.47 MB install that bundles its language models. All three run offline once installed. Timings depend on the machine.

6 Discussion

What the results support. Two things. First, as shipped, pygarble's `english_fusion` profile captures about nine in ten real survey junk responses and nearly every random string while flagging about one tweet in fifteen, from a one-megabyte install with no dependencies. Neither baseline both captures more survey junk and flags fewer tweets: the Markov baseline flags far fewer tweets but misses more than a quarter of the survey junk, and lingua captures almost all junk but flags more tweets and needs a three-hundred-megabyte install. Second, the ensemble is tunable from a small labelled sample in seconds, though tuning trades recall for precision rather than improving every endpoint. The learned weights lower the tweet false-positive rate from 6.9 % to 4.9 %, below the profile's in each of the five draws, and keep capture about level on average (89.7 % against 89.6 %), but recall falls (97.6 % against 99.1 %). The draws vary: on the primary draw both capture and recall are below the profile's. The weights explain the result at the extremes: `word_lookup` (1.00), `zipf_conformity` (0.81), `entropy_based` (0.78) and `hex_string` (0.76) get the largest mean weights, and `ngram_rank` (0.10), `control_characters` (0.04) and `mojibake` (0.03) get little or none; for the two encoding checks that fits the dataset labelling encoding damage as clean. Individual weights vary widely between draws, so only the extremes are stable. A user who disagrees with that labelling turns those checks back on; nothing is hidden inside a model.

Where pygarble is beaten. The Markov baseline has the lowest false-positive rate on tweets, 0.3 % against 6.8 %; when wrongly flagging a real post costs much more than missing junk, it is the better default. Lingua captures more survey junk, 98.9 % against 89.5 %, under its aggressive policy. The default `english` profile is not suitable for informal text as shipped: it flags a third of tweets, its encoding checks 28.5 % and its Markov member alone 17.8 %.

Guidance. For screening survey or form free text with no labelled data, use `english_fusion`; add a minimum-length rule if one- and two-character answers count as junk. With a labelled sample of the target domain (the study used 2,697 rows), learn a weight per strategy and a threshold as in Section 5.3: it takes seconds, needs no model, and gave a lower false-positive rate and higher

precision than `english_fusion` in every draw, but lower recall in four of five, so check recall on held-out labelled rows before adopting the learned weights. Without labels, start from a profile and remove the strategies whose target the domain does not treat as junk. When false positives are costly, prefer the Markov baseline at default settings, or measure a threshold on labelled rows of your own data. For multilingual input, route by language first; a language identifier answers that question, and it is not a junk detector.

7 Limitations

Both datasets are English and short. We measure how much labelled junk is caught and how many real tweets are wrongly flagged; we do not measure behaviour on clean survey answers, bot spam, other languages or long documents, and the results should not be read beyond that scope. Survey Junk has only junk, so it measures capture and nothing about false positives; Tweets vs. Random Strings has random strings as its only junk, which are easier than real junk. There are two baselines and no neural one. The main comparison runs every method at its default decision, so it says how the tools behave out of the box. Only pygarble is tuned; a tuned baseline might narrow the gap. The tuning study was added after the main results were seen and learns from labelled rows of both datasets; its evaluation rows are held out, but its samples come from the same two datasets, so the learned weights fit them and not junk in general. A comparison at a matched false-positive rate, with thresholds fitted on a development split, was computed in an earlier version of the protocol and removed from the paper's scope; in it the Markov baseline captured more survey junk than `english_fusion`. It remains in the repository history.

8 Reproducibility

The benchmark code, the frozen protocol with its dated deviations file, and the results are in the repository under `paper/`. The manifest records the SHA-256 hash of each raw file (for Survey Junk, beginning `091d5d3895f035d5`) and every row id; no dataset text is committed. Per-row scores and decisions for every method are committed, so every number in this paper can be recomputed without the raw data:

```
make benchmark-verify PYTHON=<venv>/bin/python
make preprint PYTHON=<venv>/bin/python
```

The first command recomputes the metrics from the saved predictions, and the tuning results from the cached strategy scores, the recorded tuning rows and the learned weights, and fails on any difference. The second regenerates the figure, the tables and the number macros this paper is typeset from, then builds the PDF. Rescoring from the raw files is described in the benchmark README; scores are deterministic.

AI assistance

The authors designed the study: they chose the datasets, the baselines, the endpoints, the default-decision protocol and the tuning design, and made every decision recorded in the deviations log. Claude Code agents running Claude Opus 5.5, Claude Sonnet 5.5 and Claude Fable 5.1 implemented the benchmark and tuning scripts, executed them, generated the tables and figure from the saved results, and drafted and copy-edited the text under the authors' direction. The authors reviewed the code, re-ran the verification script, checked the reported numbers against the saved predictions and revised the manuscript, and they are responsible for the methods, results and conclusions. Commit trailers in the repository record the model behind each commit.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

[2] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems 24*, pages 2546–2554, 2011.

[3] William B. Cavnar and John M. Trenkle. N-gram-based text categorization. In *Proceedings of the Third Annual Symposium on Document Analysis and Information Retrieval (SDAIR)*, pages 161–175, 1994.

[4] Ronald A. Fisher. *Statistical Methods for Research Workers*. Oliver and Boyd, Edinburgh, 1925.

[5] Michael Hucka. Nostril: A nonsense string evaluator written in Python. *Journal of Open Source Software*, 3 (25):596, 2018. doi: 10.21105/joss.00596.

[6] Madhur Jindal. autonlp-Gibberish-Detector-492513457. Hugging Face model, `https://huggingface.co/madhurjindal/autonlp-Gibberish-Detector-492513457`, 2021. DistilBERT classifier; year as given in the model card's citation; accessed 2026-10-11.

[7] Aaron Loo. gibberish-detector. `https://github.com/domanchi/gibberish-detector`, 2021. Version 0.1.1 on PyPI; a port of `https://github.com/rrenaud/Gibberish-Detector`; accessed 2026-10-11.

[8] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis and insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2021.

[9] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[10] Ujjwal Singh Rao and Ricky Singh. pygarble: local screening of text for secrets, PII, profanity and gibberish. `https://github.com/brightertiger/pygarble`, 2026. Version 0.11.0, commit e3135ed.

[11] Sowmya15. gibberish_multilingual. Hugging Face dataset, `https://huggingface.co/datasets/Sowmya15/gibberish_multilingual`, 2024. Apache-2.0; accessed 2026-10-11.

[12] Peter M. Stahl. Lingua: the most accurate natural language detection library for Python. `https://github.com/pemistahl/lingua-py`, 2026. Package lingua-language-detector, version 2.2.0; accessed 2026-10-11.

[13] John Wackerow. Gibberish Text Classification. Kaggle dataset, `https://www.kaggle.com/datasets/johnwdata/gibberish-text-classification`, 2019. File `Gibberish.xls`; licence listed as Unknown; accessed 2026-10-11.

[14] Jacob Ziv and Neri Merhav. A measure of relative entropy between individual sequences with application to universal classification. *IEEE Transactions on Information Theory*, 39(4):1270–1279, 1993.

Table 6: The seven named profiles, their members and voting rules.

Profile	Members	Voting
`english` (default)	`markov_chain`, `log_likelihood_ratio`, `word_anomaly`, `mojibake`, `keyboard_adjacency`, `control_characters`	any
`english_extended`	the six `english` members, `pattern_matching`, `local_anomaly`, `repetition`	any
`legacy`	`markov_chain`, `log_likelihood_ratio`, `word_anomaly`	any
`corruption`	`mojibake`, `control_characters`	any
`spoofing`	`unicode_script`	any
`llm_output`	`repetition`, `control_characters`, `mojibake`, `local_anomaly`	any
`english_fusion`	`word_lookup`, `log_likelihood_ratio`, `cross_parsing`	Fisher

any: the text is flagged when any applicable member scores 0.5 or more. *Fisher*: each member's score is read as a tail p-value against synthetic English, the p-values are combined by Fisher's method, and the text is flagged at a combined $p \leq 0.001$.

A Profiles

Table 6 lists the seven named profiles with their members and voting rules.

B Strategies

Table 7 lists every strategy by family with what it measures and which of the two measured profiles it belongs to.

C Learned weights

Table 8 lists the weight each strategy received in the tuning study of Section 5.3, as the mean over the five draws and in the primary draw, whose threshold is 0.16. A weight of zero switches the strategy off.

Table 8: Learned weights, sorted by the mean over five draws.

Strategy	Mean weight	Seed 20261011
`word_lookup`	1.00	1.00
`zipf_conformity`	0.81	0.25
`entropy_based`	0.78	1.00
`hex_string`	0.76	0.85
`permutation_test`	0.73	0.25
`letter_position`	0.71	0.25
`log_likelihood_ratio`	0.69	0.90
`word_anomaly`	0.67	1.00
`ngram_frequency`	0.60	0.65
`vowel_pattern`	0.57	0.65
`primed_compression`	0.52	0.95
`bigram_probability`	0.51	0.55
`consonant_sequence`	0.51	0.50
`unicode_script`	0.50	0.45
`keyboard_adjacency`	0.48	0.60
`markov_chain`	0.48	0.80
`repetition`	0.42	0.30
`affix_detection`	0.41	0.15
`rare_trigram`	0.40	0.70
`function_word_density`	0.39	0.45
`symbol_ratio`	0.38	0.85
`letter_frequency`	0.34	0.75
`local_anomaly`	0.33	0.05
`cross_parsing`	0.31	0.60
`pattern_matching`	0.30	0.00
`keyboard_pattern`	0.28	0.35
`pronounceability`	0.28	0.65
`word_collocation`	0.25	0.05
`vowel_ratio`	0.23	0.05
`ngram_rank`	0.10	0.05
`control_characters`	0.04	0.00
`mojibake`	0.03	0.00

Table 7: The 32 strategies of pygarble by family. A strategy *fires* when its default configuration scores at least 0.5. Profile: E, member of `english` (the default); F, member of `english_fusion`. Words "in the dictionary" are those in the embedded list of 49,330 English words; checks restricted to non-dictionary words also skip short all-capital acronyms and structured tokens such as URLs and numbers.

Strategy	Profile	What it measures and when it fires
Character statistics		
`vowel_ratio`		Share of vowels in words of three or more letters and the longest consonant run in a word; fires when vowels are under about 7.5% or over about 82% of letters, or a word has six consonants in a row.
`entropy_based`		Blend of low Shannon entropy of the letters (40%) and low mean English bigram log-probability (60%); fires when the blend reaches 0.5, as on repetitive or improbable letter strings.
`consonant_sequence`		Longest run of consonants inside a word, with *y* counted as a vowel and short all-capital acronyms skipped; fires on a run of seven or more.
`letter_frequency`		Chi-squared distance of the letters of non-dictionary words from English letter frequencies, plus rare-letter excess, missing common letters and one dominant letter; fires at a per-letter distance of 2 or two extreme signs.
`vowel_pattern`		Runs of consecutive vowels within words; fires on a run of five or more vowels that is not on a short list of valid long runs such as *ueuei*.
`symbol_ratio`		Share of non-space characters that are neither letters nor digits, after collapsing runs of one repeated symbol such as rulers; fires when symbols are more than half of the text.
`letter_position`		Non-dictionary, non-capitalised words that end in a letter English words never end in or start with a pair English words never start with; fires when a quarter of these checks fail across two or more violations or words.
`pronounceability`		English phonotactics of non-dictionary, non-capitalised words of four or more letters; fires on two forbidden consonant clusters at word edges, mostly invalid word-initial clusters, or almost no vowels.
English character models		
`markov_chain`	E	Mean log-probability per letter transition of the non-dictionary words under an English character bigram model; fires when it falls below −3.5.
`bigram_probability`		Share of within-word letter pairs on a curated list of pairs that English essentially never uses; fires when that share reaches a quarter.
`ngram_frequency`		Share of the letter trigrams of non-dictionary words that are on a list of common English trigrams; fires when fewer than 30% are.
`rare_trigram`		Share of within-word letter trigrams on a curated list of combinations that English essentially never uses; fires when that share reaches 15%.
`log_likelihood_ratio`	E, F	Mean log-likelihood ratio per letter pair of all words, English bigram model against uniform letters; fires when the mean falls below −1, so the text fits English clearly worse than random letters.
Word-level checks		
`word_lookup`	F	Share of words of two or more letters that are not in the dictionary, unknown capitalised words counting half; fires when half or more are unknown.
`word_anomaly`	E	Share of words of four or more letters that are non-dictionary and have mean bigram log-probability below −4.6; fires when a quarter of the words are anomalous, so one mashed word in four suffices.
`function_word_density`		Share of English function words such as *the*, *of* and *is*; fires only when ten or more words contain none and the text is not mostly Title Case.
`affix_detection`		Share of words of four or more letters that carry a common English prefix or suffix on a stem with a vowel; fires only on twenty or more such words when fewer than about one in six do.
`word_collocation`		Adjacent word pairs found in a list of common English pairs such as *of the*; fires when twenty or more words, not mostly Title Case, contain no such pair, or fifteen or more have under 2% hits.
`zipf_conformity`		Type-token ratio and share of once-only words in texts of thirty or more words; fires on a flat distribution only when the text also has no function words and mostly non-dictionary words.

Table 7: The 32 strategies of pygarble by family (continued).

Strategy	Profile	What it measures and when it fires
Structure and pattern checks		
`repetition`		Runs of one repeated letter, repeated two- or three-character units, one word dominating the text or a repeating cycle of words, and low character diversity; fires, for example, on a letter repeated five times.
`keyboard_pattern`		Letter trigrams of non-dictionary words that follow keyboard sequences, a shortage of common English trigrams, and a two-letter unit repeated three times inside one word; fires when keyboard trigrams reach 15%, on the repeated unit, or when a text of twenty or more trigrams has almost none of the common English trigrams.
`keyboard_adjacency`	E	Chains of physically adjacent QWERTY keys within words; fires when a word of five or more characters is a straight row walk, or a non-dictionary word has a six-key adjacent chain covering at least 60% of it, or six same-row keys with at most one vowel.
`hex_string`		Hexadecimal and encoded data; fires when the whole text is an MD5 or SHA-256 hash, a base64-like string or a UUID, on a run of sixteen or more hex digits including a letter, or on a long token that is mostly hex digits.
`pattern_matching`		Built-in regular expressions (a letter four or more times in a row, five-key keyboard-row walks, five consonants in a row in a non-dictionary word, alternating letter pairs) plus optional user patterns; fires on any of these, while runs of symbols, capitals or digits only add weight.
`local_anomaly`		Severely improbable non-dictionary words inside otherwise ordinary text; fires when one such word has eight or more letters or two fall within any four consecutive words.
Encoding and script checks		
`mojibake`	E	Signatures of text decoded with the wrong encoding; fires on a replacement character U+FFFD, a known sequence such as Ã© or â€™, double encoding, or a dense run of such byte pairs.
`control_characters`	E	Raw control characters other than tab, carriage return and newline, lone surrogates, U+FFFD, and runs of more than eight combining marks; fires on any one of them.
`unicode_script`		Mixing of Unicode scripts; fires on any word of three or more letters that mixes two of the Latin, Cyrillic and Greek scripts, as homoglyph spoofing does, or on text that uses more than two scripts.
English-reference window statistics (each fires only when its median over windows of up to 127 characters lies well beyond the 99th percentile of synthetic English of similar length)		
`cross_parsing`	F	Number of pieces needed to spell the text from the longest substrings of a reference built from frequent English words (Ziv–Merhav cross parsing [14]); invented or mashed text breaks into many short pieces.
`primed_compression`		Compressed size per character when deflate is primed with frequent English words as a preset dictionary; English compresses well against it and invented text does not.
`ngram_rank`		Out-of-place distance between the frequency ranks of the text's one- to three-letter sequences and their ranks in English (Cavnar–Trenkle [3]).
`permutation_test`		How much likelier the text's letter order is under an English bigram model than eight fixed shuffles of the same letters; catches mash and random letters but often misses pronounceable invented words.